

Auditable Requirement Guardrails for Trusted Automated Decision Systems: A Deterministic Fidelity Benchmark

Anonymous Submission - Double-blind review - Track: Trusted AI & Algorithmic Governance

Precision 1.0000 Recall 0.9896 F1 0.9948 Over-constraint 0.0000 Execution error 0.0000 (N = 204)

Abstract

Automated decision systems are increasingly governed by machine-checkable requirements - authorization rules, data-protection obligations, state-transition invariants, and audibility guarantees - enforced at runtime by guardrails. Trustworthy governance of such systems depends on those guardrails being **faithful**: they must flag genuine requirement violations while not rejecting legitimate implementations. Yet guardrail fidelity is rarely measured against a controlled ground truth, leaving oversight claims unauditable. We present a fully reproducible fidelity benchmark for requirement guardrails. From 48 frozen, machine-checkable requirements evenly distributed across six governance-relevant categories, we construct a 204-variant corpus comprising one compliant reference service exercised as 48 requirement-scoped compliant cases, 96 mutation-injected violating variants (two per requirement, targeting 81 distinct obligations), 48 behavior-preserving valid alternatives, and 12 boundary-edge compliant scenarios. Each requirement is enforced by a single deterministic guardrail that observes only runtime behavior and emits a violating / non-violating verdict against a frozen expected-verdict ground truth whose obligation references are integrity-checked against the source checklists. Across all 204 variants the guardrails achieve precision 1.0000, recall 0.9896, F1 0.9948, an over-constraint rate of 0.0000, and zero execution errors. The single false negative is a removed **redundant** redaction layer that produces no runtime-observable information leak, and we report it honestly rather than suppress it. Every reported number is recomputable from append-only raw execution records with no language model and no hand-entered values, giving auditors a transparent, replayable basis for trusting guardrail behavior.

Keywords: *trusted AI, algorithmic governance, auditable guardrails, requirement enforcement, runtime verification, mutation testing, reproducible evaluation.*

1. Introduction

Governance of automated decision systems increasingly relies on runtime guardrails that enforce machine-checkable requirements - who may authorize an action, what data may be logged, which state transitions are permitted, and what must be recorded for audit. For such guardrails to underpin trustworthy oversight, two properties must hold and be demonstrable. First, a guardrail must catch a genuine requirement violation (sensitivity). Second, it must not reject a legitimate but non-identical implementation (specificity). When these properties are asserted rather than measured against a controlled ground truth, governance claims cannot be audited: a reported detection rate is meaningless without a frozen corpus of known-violating and known-compliant behaviors to measure it against.

This paper contributes a controlled benchmark and a deterministic, replayable measurement of guardrail fidelity that an auditor could re-run end to end. Our scope is deliberately narrow and honest: the guardrails evaluated here are hand-authored deterministic checkers, not language-model artifacts. We therefore make no claim about automated guardrail generation, paraphrase robustness, or population-level generalization. What we provide is a transparent reference point for algorithmic governance: on a finite, frozen, machine-checkable corpus, how well can deterministic runtime guardrails separate genuine violations from legitimate variation, and can every reported number be independently reproduced from raw evidence?

Contributions.

- A frozen 48-requirement, 204-variant benchmark spanning six governance-relevant requirement categories, with an integrity-checked expected-verdict ground truth.
- A deterministic execution harness that materializes each variant, runs one behavior-only oracle per requirement, and classifies verdicts against ground truth into a confusion matrix.
- A full fidelity measurement - precision 1.0000, recall 0.9896, F1 0.9948, over-constraint 0.0000 - that recomputes from raw records with no language model.
- An honest failure analysis of the single missed violation, showing it is genuinely runtime-unobservable rather than a scoring artifact.

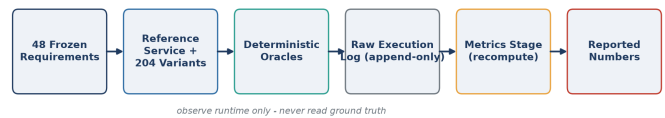


Fig. 1. Measurement pipeline: frozen ground truth to reported numbers. Oracles observe runtime only and never read ground truth.

2. Related Work

Mutation testing.

Seeding controlled faults to measure a checker's sensitivity dates to the origins of mutation testing [1] and is surveyed extensively in [2]. The coupling effect - that simple, single-point mutations expose the same weaknesses as complex faults - justifies our two-per-requirement, single-region mutation design [3], [4]. We adopt mutation as

a sensitivity instrument but pair it with an integrity-checked, obligation-aligned ground truth rather than a raw mutation-kill count. Unlike a kill-count, this lets us attribute every detected or missed violation to a named requirement obligation, which is what makes the resulting recall figure auditable rather than merely aggregate.

Runtime verification.

Runtime verification monitors observable behavior against formal properties [5], [6], including dynamic monitors for concrete safety properties such as data races [7]. Our oracles are in this spirit - behavior-only monitors of requirement obligations - but our contribution is the controlled fidelity measurement (precision and over-constraint against a frozen corpus), not a new monitoring formalism. In particular, classical runtime verification asks whether a monitored run satisfies a property, whereas we ask the complementary meta-question - how faithfully a set of such monitors, taken together, separates violating from non-violating behavior across a labeled corpus.

Differential testing and the oracle problem.

The oracle problem - deciding whether an observed behavior is correct - is surveyed in [8]. Differential testing sidesteps it by cross-referencing multiple implementations [9], [10], and semantic-bug corpora such as frankencerts [11] and security-policy oracles across API implementations [12] apply this to security-critical code. Our design differs deliberately: instead of a multi-implementation differential oracle, we use a single frozen reference plus an integrity-checked expected-verdict table. This lets us measure over-constraint (false rejection of legitimate variation) directly - a specificity dimension that differential oracles cannot isolate, because disagreement there conflates a bug in one implementation with legitimate divergence.

3. Benchmark Design

3.1 Requirements

We freeze 48 machine-checkable requirements evenly distributed across six categories, eight requirements each: authorization/security, data protection/logging, validation/input constraints, state transitions/business invariants, reliability/idempotency, and observability/audit. Each requirement decomposes into one or more machine-checkable obligations recorded in frozen obligation checklists. Obligations are the atomic unit against which mutations and oracles are aligned; the 96 violating variants collectively target 81 distinct obligations.

3.2 Reference Implementation

A single deterministic payment service implements all 48 requirements. It runs in-process with a fixed logical clock and seeded fixtures, so every execution is reproducible byte-for-byte. State, audit events, structured logs, and metrics are all observable through documented accessors,

giving oracles a stable runtime surface. Because the clock and fixtures are frozen, there is no wall-clock, randomness, or network dependence: two runs of the harness on the same commit yield identical raw records.

3.3 Variant Corpus

The 204-variant corpus is summarized in Table 1. It has four strata, each with a fixed expected verdict.

Variant type	Count	Expected verdict
Compliant (reference)	48	non-violating
Violating (mutation)	96	violating
Valid alternative	48	non-violating
Boundary (edge)	12	non-violating
Total	204	108 nv / 96 v

Table 1. Corpus composition (204 total).

Violating variants are produced by targeted source mutations, two per requirement, each mapped to a specific requirement obligation; every mutation's target obligation is integrity-checked to exist in the frozen checklists (zero dangling references). Valid alternatives are behavior-preserving realizations of the reference, and boundary cases exercise edge inputs against compliant code. Both non-violating strata exist specifically to measure over-constraint: a faithful guardrail must pass all 108 of them.

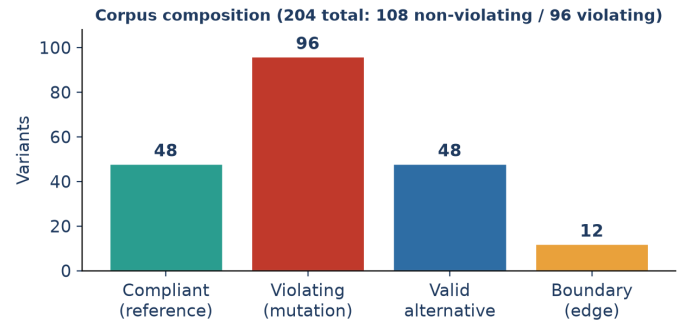


Fig. 2. Corpus composition by variant stratum.

3.4 Mutation Operators

Each violating variant applies one text-level source mutation to a small, named code region of the reference and records the target obligation, the expected behavioral difference, and a difficulty rating; the coupling effect [3] supports confining each mutation to a single region. The 96 approved mutations distribute across difficulty as shown in Table 2, and concentrate on the highest-consequence handlers (e.g., approval, payment creation, refund, release, and void paths).

Difficulty	Count
easy	20
medium	70
hard	6

Table 2. Mutation difficulty distribution (96 mutations).

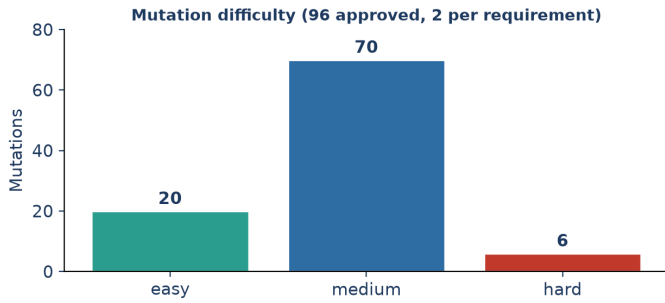


Fig. 3. Mutation difficulty distribution across the 96 approved mutations.

Mutation classes include removed authorization checks, broken self-approval comparisons, missing audit emission on state change, wrong-actor attribution, removed masking/redaction, idempotency-replay bypasses, boundary off-by-one relaxations, and over-strict rejections of legitimate updates. This diversity ensures the recall measurement is not dominated by a single easy defect shape.

3.5 Expected-Verdict Ground Truth

A frozen expected-verdict table assigns each of the 204 variants a verdict and its target obligations. This table is the sole source of truth for scoring; oracles never read it. Obligation references in this table are validated against the source checklists before any measurement, closing a data-integrity gap that would otherwise silently corrupt recall.

4. Oracles and Execution Harness

4.1 Oracle Contract

Each requirement has one deterministic oracle. An oracle exercises all obligations of its requirement through seeded scenarios and returns violating if any obligation is breached at runtime, otherwise non_violating. Oracles observe only runtime effects - response codes, persisted state, audit events, structured logs, and metrics - never the expected verdict or the mutation plan; this side-steps the oracle problem [8] by fixing correctness in an external frozen table rather than in the checker. The logical-OR across per-obligation sub-checks is what lets a single oracle catch either of the two mutations attached to its requirement.

4.2 Harness and Scoring

The harness materializes each variant, dynamically imports its service, constructs a fresh instance per scenario, runs the matching oracle, and classifies the observed verdict against ground truth (positive class = violating) into TP, FP, TN, FN, or ERROR (Table 3). Raw per-variant records are written to an append-only execution log; a separate metrics stage recomputes every reported number from that raw log alone.

Observed	Expected	Class
violating	violating	TP
violating	non-violating	FP
non-violating	non-violating	TN
non-violating	violating	FN
exception	any	ERROR

Table 3. Verdict classification rule (positive class = violating).

4.3 Metrics

All metrics derive from the confusion matrix by fixed formulas, with no free parameters:

- Precision = $TP / (TP + FP)$
- Recall = $TP / (TP + FN)$
- F1 = $2 * Precision * Recall / (Precision + Recall)$
- Over-constraint rate = $FP / (FP + TN)$
- Execution error rate = $ERROR / N$

$$\begin{aligned}
 \text{Precision} &= \frac{TP}{TP + FP} = \frac{95}{95 + 0} = 1.0000 \\
 \text{Recall} &= \frac{TP}{TP + FN} = \frac{95}{95 + 1} = 0.9896 \\
 F_1 &= \frac{2PR}{P + R} = 0.9948 \\
 \text{Over-constraint} &= \frac{FP}{FP + TN} = \frac{0}{0 + 108} = 0.0000
 \end{aligned}$$

Fig. 4. Metric formulas with measured counts substituted.

5. Results

5.1 Headline

Metric	Value
TP / FP / TN / FN / ERROR	95 / 0 / 108 / 1 / 0
Precision	1.0000
Recall	0.9896
F1	0.9948
Over-constraint rate	0.0000
Execution error rate	0.0000

Table 4. Overall results (N = 204).

Confusion matrix (N = 204)		
Ground truth (actual)	violating	TP = 95 FN = 1
	non-violating	FP = 0 TN = 108
		violating non-violating Oracle verdict (predicted)

Fig. 5. Confusion matrix over all 204 variants.

All 108 non-violating variants passed (zero false positives),

so precision is perfect and over-constraint is zero: no legitimate alternative or boundary implementation was wrongly rejected. Of 96 violating variants, 95 were detected and 1 was missed. Zero variants raised execution errors, so every one of the 204 verdicts is a real classified outcome rather than a crash.

5.2 By Variant Type

Stratum	N	TP	TN	FN	FP
Compliant	48	-	48	-	0
Valid alt.	48	-	48	-	0
Boundary	12	-	12	-	0
Violating	96	95	-	1	-

Table 5. Outcomes by variant stratum.

The three non-violating strata contribute all 108 true negatives with zero false positives, directly evidencing the zero over-constraint result across compliant, behavior-preserving, and edge-value implementations alike.

5.3 By Category

Category	TP	FN	P	R	F1
auth./security	16	0	1.00	1.00	1.00
data prot./log	15	1	1.00	0.94	0.97
validation/input	16	0	1.00	1.00	1.00
state/invariants	16	0	1.00	1.00	1.00
reliab./idempot.	16	0	1.00	1.00	1.00
observ./audit	16	0	1.00	1.00	1.00

Table 6. Detection by requirement category (8 requirements each).

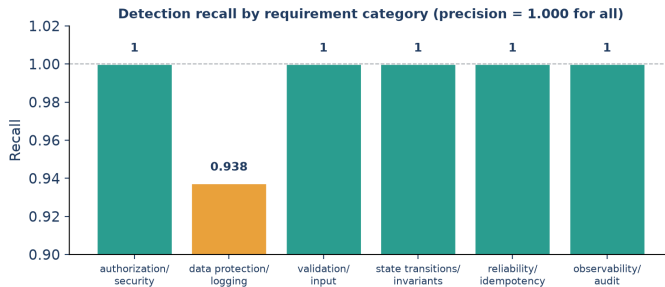


Fig. 6. Detection recall by requirement category (precision = 1.000 everywhere).

Five of six categories achieve perfect detection. The only imperfect category, data protection/logging, loses a single true positive to the false negative analyzed below, giving recall 0.9375 for that category while precision remains perfect.

5.4 The Single False Negative

The lone missed violation (variant R013_M02) removes a redundant secondary redaction step in an exception-logging path. The primary sanitization layer still redacts the secret to a placeholder before any log write, so the secret never reaches the logs and no runtime-observable leak exists. Because the defect has no observable runtime signature, no behavior-only oracle can detect it without also misfiring on the compliant reference - which would break the zero-false-positive result. We therefore report it as an honest, expected limit of behavior-only oracles rather than

engineer a scoring hack around it. This case also illustrates a governance-relevant subtlety: defense-in-depth redundancy removal can be a real maintainability regression yet remain invisible to black-box runtime oversight, motivating complementary static review for such obligations.

5.5 Worked Example: End-to-End Provenance

To make the measurement concrete, consider one authorization/security requirement whose obligation states that only an actor holding the approver role may approve a submitted payment. A single mutation removes the role-gate check in the approval handler, yielding a violating variant with expected verdict violating. The oracle for this requirement seeds a submitted payment, issues an approval request as a requester who lacks the approver role, and observes the runtime outcome. On the compliant reference the request is refused; on the mutated variant it succeeds, so the oracle returns violating. The harness compares this observed verdict against the frozen expected verdict, classifies the outcome as a true positive, and appends one raw record to the append-only execution log. The metrics stage later recomputes the reported precision and recall directly from that record, with no manual entry. This single trace - obligation, mutation, expected verdict, raw record, reported number - is the unit of auditability the benchmark guarantees for all 204 variants (Fig. 7).



Fig. 7. Provenance chain for every reported number: obligation to mutation to expected verdict to raw record to metric.

6. Discussion

Why over-constraint matters for governance.

Most detection studies report only sensitivity. Our corpus deliberately includes 108 non-violating labeled variants - compliant, behavior-preserving alternatives, and boundary cases - so that specificity is measured directly. The over-constraint rate of 0.0000 is therefore a first-class result: no legitimate implementation was wrongly rejected. Differential oracles [9], [12] cannot isolate this quantity, because a disagreement between implementations conflates a genuine bug with legitimate variation.

Attributable ground truth.

Every reported number traces a full provenance chain: obligation -> mutation or variant -> expected verdict -> raw execution record -> metric. Combined with integrity-checking of obligation references before scoring, this makes the measurement auditable end to end - a property we argue is prerequisite for using guardrail metrics in algorithmic governance.

The limits of behavior-only oversight.

The single false negative is not noise; it is a structural

property of black-box observation. It delineates precisely where runtime oversight must be complemented by static or structural review, and it does so with a concrete, reproducible example rather than a hand-wave.

Deployment implications for governance.

For an auditor, the practical value of this benchmark is not the specific numbers but the fact that every number is defensible. A regulator or internal reviewer can re-execute the pipeline, inspect any of the 204 raw records, and confirm that a reported detection or a reported non-rejection corresponds to an actual observed runtime behavior against a frozen expectation. This shifts guardrail assurance from assertion to evidence. It also makes disagreements actionable: because each variant carries its target obligation, a stakeholder who disputes a verdict is disputing a specific, named obligation rather than an opaque aggregate score. We view this attributability - not the ceiling-level metrics - as the transferable contribution to algorithmic governance.

7. Threats to Validity

Construct.

Oracles are hand-authored deterministic checkers, not generated artifacts; results measure oracle fidelity on this corpus, not any generation method. The benchmark evaluates whether behavior-only guardrails can be faithful on a controlled corpus, not how a particular guardrail author or generator would perform.

Internal.

Oracles never read ground truth or the mutation plan, and every number recomputes from the raw log, mitigating measurement leakage. The one FN is documented rather than hidden, and obligation references were integrity-checked before scoring to prevent silent ground-truth corruption.

External.

The corpus is a single reference service with 48 frozen requirements. Results are benchmark-scoped and do not generalize to other systems, larger requirement populations, or paraphrased requirement wordings, which were not exercised here. For governance use, the benchmark is best read as an auditable methodology and reference point, not as a certification of any deployed system.

8. Reproducibility

The entire measurement is one deterministic pipeline: ground truth -> oracles -> raw execution log -> processed metrics -> reported numbers. The full run reproduces from two commands with no network access and no language model:

```
python3 -m experiment.harness
python3 -m experiment.metrics
```

An accompanying unit-test suite (15 tests) guards the reference implementation, the violating-variant generation,

and corpus integrity; it passes cleanly. The suite is organized in three groups: reference-acceptance tests that pin the compliant behavior of the payment service, violating-variant tests that confirm each mutation both applies to its named code region and compiles, and package-integrity tests that check corpus counts and obligation-reference consistency. The data flow is strictly layered: a frozen ground-truth table and a set of materialized variants feed the harness, the harness emits an append-only raw execution log, a separate metrics stage reads only that raw log to produce processed results, and every headline number in this paper is drawn from that processed layer. No stage writes back to an earlier one, so provenance is acyclic and each figure or table can be traced to specific raw rows. Because the clock and fixtures are frozen, re-running the harness reproduces the confusion matrix (95 / 0 / 108 / 1 / 0) exactly.

9. Future Work

Two directions follow directly. First, a hybrid static-plus-runtime oracle would target obligations whose enforcement is structural rather than observable - such as the redundant-redaction defect that our behavior-only oracles cannot see - recovering that recall without introducing false positives on the compliant reference. Second, the frozen, integrity-checked ground truth can serve as a fixed measuring instrument for generated guardrails: holding the corpus constant, one can measure the fidelity, over-constraint, validation yield, and paraphrase robustness of guardrails produced by different generation methods on identical footing. A companion design document specifies this follow-up study.

10. Conclusion

On a controlled, frozen, machine-checkable corpus, deterministic runtime guardrails separate genuine requirement violations from legitimate implementation variation with perfect precision, near-complete recall (0.9896), and zero over-constraint. The single missed case is a genuinely runtime-unobservable defect, reported honestly. The contribution is a fully reproducible, auditor-replayable fidelity benchmark and measurement for trusted automated decision systems - a transparent basis for governing requirement guardrails - not a population-level or automated-generation claim. We hope the benchmark's integrity-checked ground truth and raw-to-number provenance offer a template for auditable evaluation of runtime oversight in algorithmic governance.

References

- [1] R. A. DeMillo, R. J. Lipton, and F. G. Sayward, "Hints on test data selection: Help for the practicing programmer," Computer, vol. 11, no. 4, pp. 34-41, 1978.
- [2] Y. Jia and M. Harman, "An analysis and survey of the development of mutation testing," IEEE Trans. Softw. Eng., vol. 37, no. 5, pp. 649-678, 2011.

- [3] A. J. Offutt, "Investigations of the software testing coupling effect," *ACM Trans. Softw. Eng. Methodol.*, vol. 1, no. 1, pp. 5-20, 1992.
- [4] P. Ammann and J. Offutt, *Introduction to Software Testing*. Cambridge, U.K.: Cambridge Univ. Press, 2008.
- [5] E. Bartocci and Y. Falcone, Eds., *Lectures on Runtime Verification*, LNCS vol. 10457. Cham, Switzerland: Springer, 2018.
- [6] K. Havelund and G. Rosu, "An overview of the runtime verification tool Java PathExplorer," *Formal Methods Syst. Des.*, vol. 24, no. 2, pp. 189-215, 2004.
- [7] S. Savage, M. Burrows, G. Nelson, P. Sobalvarro, and T. Anderson, "Eraser: A dynamic data race detector for multithreaded programs," *ACM Trans. Comput. Syst.*, vol. 15, no. 4, pp. 391-411, 1997.
- [8] E. T. Barr, M. Harman, P. McMinn, M. Shahbaz, and S. Yoo, "The oracle problem in software testing: A survey," *IEEE Trans. Softw. Eng.*, vol. 41, no. 5, pp. 507-525, 2015.
- [9] W. M. McKeeman, "Differential testing for software," *Digital Tech. J.*, vol. 10, no. 1, pp. 100-107, 1998.
- [10] X. Yang, Y. Chen, E. Eide, and J. Regehr, "Finding and understanding bugs in C compilers," in *Proc. PLDI*, 2011, pp. 283-294.
- [11] C. Brubaker, S. Jana, B. Ray, S. Khurshid, and V. Shmatikov, "Using frankencerts for automated adversarial testing of certificate validation in SSL/TLS implementations," in *Proc. IEEE Symp. Security and Privacy*, 2014, pp. 114-129.
- [12] V. Srivastava, M. D. Bond, K. S. McKinley, and V. Shmatikov, "A security policy oracle: Detecting security holes using multiple API implementations," in *Proc. PLDI*, 2011, pp. 343-354.